Prefilling-dLLM: Predictive Prefilling for Long-Context Inference in Diffusion Language Models

Jing Xiong[1] Qi Han[1] Shansan Gong[1] Yunta Hsieh[2] Boyuan Zheng[2]
Chengyue Wu[1] Chaofan Tao[1] Chenyang Zhao[3] Ngai Wong[1]

[1]The University of Hong Kong [2]University of Michigan, Ann Arbor [3]LMSYS Org

EMNLP2026 BUDAPEST · LMSYS

1. Motivation

Re-encode every step. Every denoising step re-encodes the whole prefix, so long context stays compute-bound.

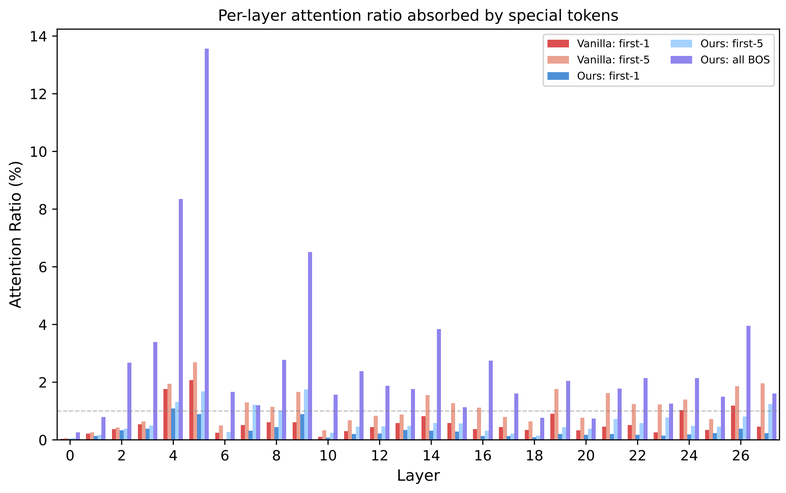
↓

Monotonic drop. Dream is exact through 4K, then accuracy falls as the gold document moves away.

↓

Score once. Position, not relevance, decides what is seen. Chunks are scored once.

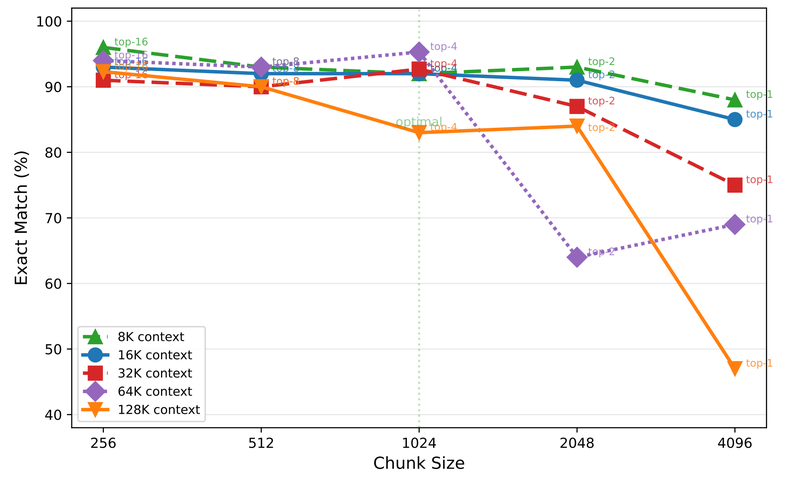
2. Findings



Finding 1

⚓ Anchors, not sinks.

One chunk BOS takes **0.30%** of the mass, not the **20–60%** of an autoregressive sink. The spikes stay **low across layers**.

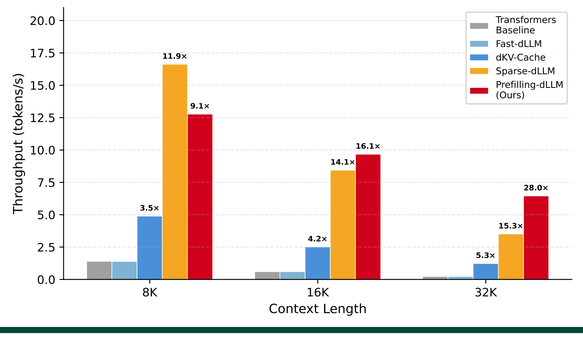


Finding 2

One chunk is not enough.

At **128K**, 256-token chunks still reach **92.3%**. A single chunk falls to **68.0%**.

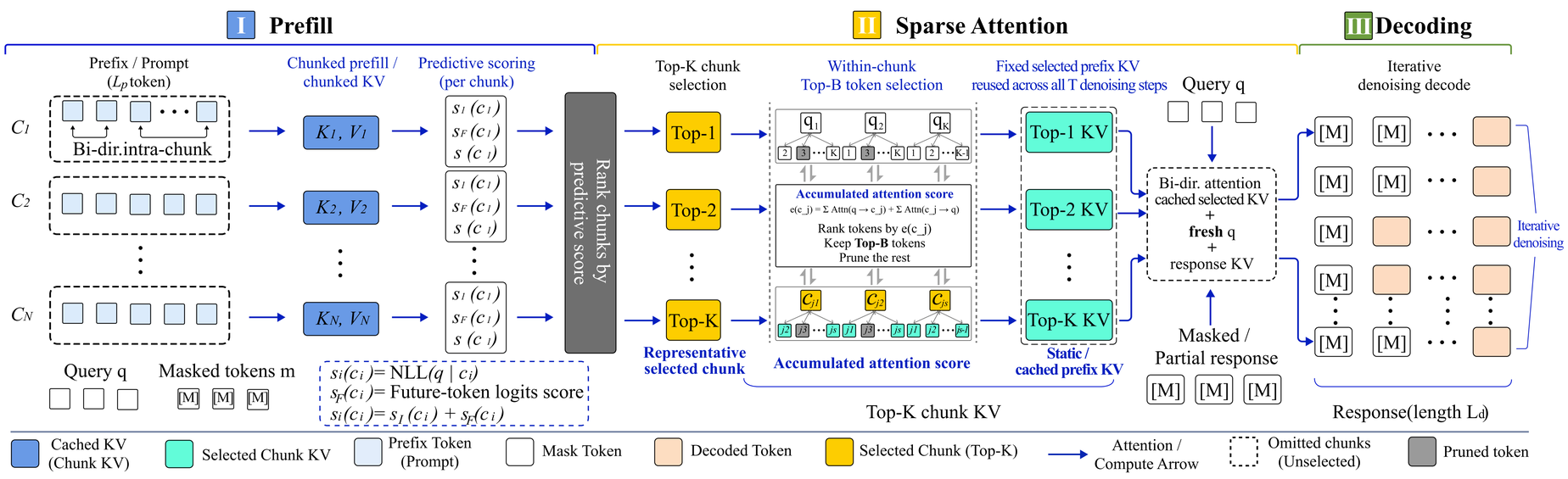
3. Efficiency



Speedup grows with length.

The cache stays near 4K: 9.1× at 8K, 16.1× at 16K, and 28.0× at 32K. Sparse-dLLM then falls to 3.51 tok/s.

5. Method



Prefill

Split the prefix into N chunks and score them in parallel. On Dream, C=1024 and top-K=4, so the budget stays near 4K.

Sparse attention

Keep the K lowest-scoring chunks and the top-B tokens in each. The response reads those tokens, not the full prefix.

Decoding

Those keys and values stay a static cache. Only the response is updated.

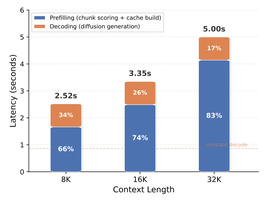
LongBench

Method	Single-doc			Multi-doc			Summ.				Few-shot		Synth.		Code		Ave.
	NQA	Qas	MF	Hot	2Wi	Mus	Gov	QMS	MN	SAM	TRE	Tri	PCnt	PRe	LCC	RBP	
Dream-7B, extrapolated to 128K via YaRN																	
YaRN (128K)	7.08	16.33	32.37	9.30	10.75	6.89	5.25	14.88	8.09	29.00	26.00	55.04	0.68	7.40	26.28	31.44	17.92
Vanilla (2K)	20.27	19.33	40.50	29.38	20.07	15.06	5.22	17.76	8.42	33.77	60.00	81.87	0.50	16.50	46.95	41.99	28.60
Fast-dLLM	9.70	18.03	36.28	11.32	11.94	7.07	5.24	13.32	7.88	31.09	46.00	70.61	0.27	3.50	24.62	29.23	20.38
Fast-dLLM v2	**27.50**	**33.25**	**49.76**	**46.00**	**35.92**	**22.90**	4.28	17.53	6.20	31.38	43.50	78.28	**4.00**	**88.50**	10.41	7.12	31.66
Sparse-dLLM	3.19	9.45	20.25	7.04	6.11	4.99	5.27	5.92	**8.13**	13.04	0.00	10.96	0.23	2.62	17.01	22.46	8.54
dKV-Cache	3.57	6.60	18.33	7.02	6.34	4.44	5.32	6.07	7.84	7.71	0.00	7.17	0.47	1.60	14.74	19.92	7.32
Ours, inter	12.14	18.28	38.26	15.32	16.13	9.75	5.40	**17.74**	7.70	29.24	46.00	70.68	0.00	13.75	21.49	30.23	22.01
Ours, +intra	19.90	26.72	46.40	36.22	22.10	19.96	**5.58**	17.52	8.02	**32.96**	**64.50**	**92.11**	1.54	76.02	**25.90**	**57.98**	**34.59**
UltraLLaDA, native 128K																	
Vanilla (128K)	17.04	22.71	33.20	15.98	16.71	12.20	5.38	17.30	8.00	37.10	79.50	90.53	1.48	96.08	67.77	54.97	36.00
Fast-dLLM	**22.03**	21.58	37.31	17.31	16.97	11.04	5.30	17.71	8.20	33.18	79.50	90.53	1.36	94.47	66.08	53.10	35.98
Sparse-dLLM	21.30	23.16	35.43	19.08	19.27	**15.39**	5.35	17.51	8.26	35.37	77.50	91.43	0.73	**97.58**	66.89	52.57	36.68
dKV-Cache	17.86	22.36	32.99	17.89	18.39	14.25	**5.41**	17.18	8.04	**36.30**	**80.50**	90.53	1.48	94.88	**67.96**	54.61	36.29
Ours, inter	12.27	22.40	**39.94**	**22.21**	**23.48**	9.64	5.33	17.85	**8.48**	34.33	67.00	88.16	2.38	90.91	65.10	**62.67**	35.64
Ours, +intra	17.07	**24.71**	36.07	21.57	17.87	14.30	5.21	**18.12**	8.46	34.43	77.50	**92.50**	**2.50**	96.91	66.92	58.20	**37.02**

InfiniteBench

Method	Passkey	Number	KV	LongBook	Math	Code	Ave.
Dream-7B, extrapolated to 128K via YaRN							
YaRN (128K)	0.00	0.00	0.00	6.11	3.43	7.87	2.90
Vanilla (2K)	1.69	2.54	0.00	35.81	34.57	0.00	12.44
Fast-dLLM	0.17	3.05	0.00	8.73	3.43	8.38	3.96
Fast-dLLM v2	27.12	27.12	0.00	57.21	24.29	46.19	30.32
Sparse-dLLM	39.32	25.59	0.00	1.31	3.43	3.81	12.24
dKV-Cache	0.51	0.00	0.00	5.68	4.00	3.05	2.21
Ours	**95.42**	**70.00**	0.00	53.71	24.57	**18.02**	**43.62**
UltraLLaDA, native 128K							
Vanilla (128K)	12.99	2.45	0.00	8.86	10.43	0.00	5.79
Fast-dLLM	**16.10**	6.61	0.00	7.86	14.29	0.25	7.52
Sparse-dLLM	8.14	2.37	0.00	5.24	7.43	**1.52**	4.12
dKV-Cache	5.82	6.75	0.00	8.30	16.00	0.00	6.15
Ours	13.22	**30.00**	0.00	**14.41**	**26.00**	**1.52**	**14.19**

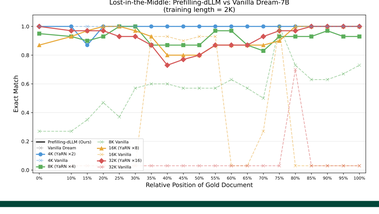
Finding 3



Cache once.

The prefix is scored once and leaves the decode loop. Decode stays near **0.86 s**, while prefill is paid once: **1.66 s, 2.49 s, and 4.14 s**.

Finding 4



Sparse can win.

On both models the sparse cache beats the **dense baseline**. Far positions stay flat. At **32K**, vanilla has collapsed.